# VISVESVARAYA TECHNOLOGICAL UNIVERSITY

**"JnanaSangama", Belgaum -590014, Karnataka.**

**LAB REPORT**
**on**

# Machine Learning (23CS6PCMAL)

***Submitted by***

**Crevan Neil Fernandes (1BM23CS082)**

***in partial fulfillment for the award of the degree of***

**BACHELOR OF ENGINEERING**
***in***
**COMPUTER SCIENCE AND ENGINEERING**

**B.M.S. COLLEGE OF ENGINEERING**
**(Autonomous Institution under VTU)**
**BENGALURU-560019**
**Jan-2025 to May-2026**

# B.M.S. College of Engineering,

**Bull Temple Road, Bangalore 560019**

(Affiliated To Visvesvaraya Technological University, Belgaum)

## Department of Computer Science and Engineering

## CERTIFICATE

This is to certify that the Lab work entitled "Machine Learning (23CS6PCMAL)" carried out by **Crevan Neil Fernandes (1BM23CS082),** who is a bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

| | |
|---|---|
| Saritha A. N<br>Assistant Professor<br>Department of CSE,<br>BMSCE | Dr. Kavitha Sooda<br>Professor & HOD<br>Department of CSE, BMSCE |

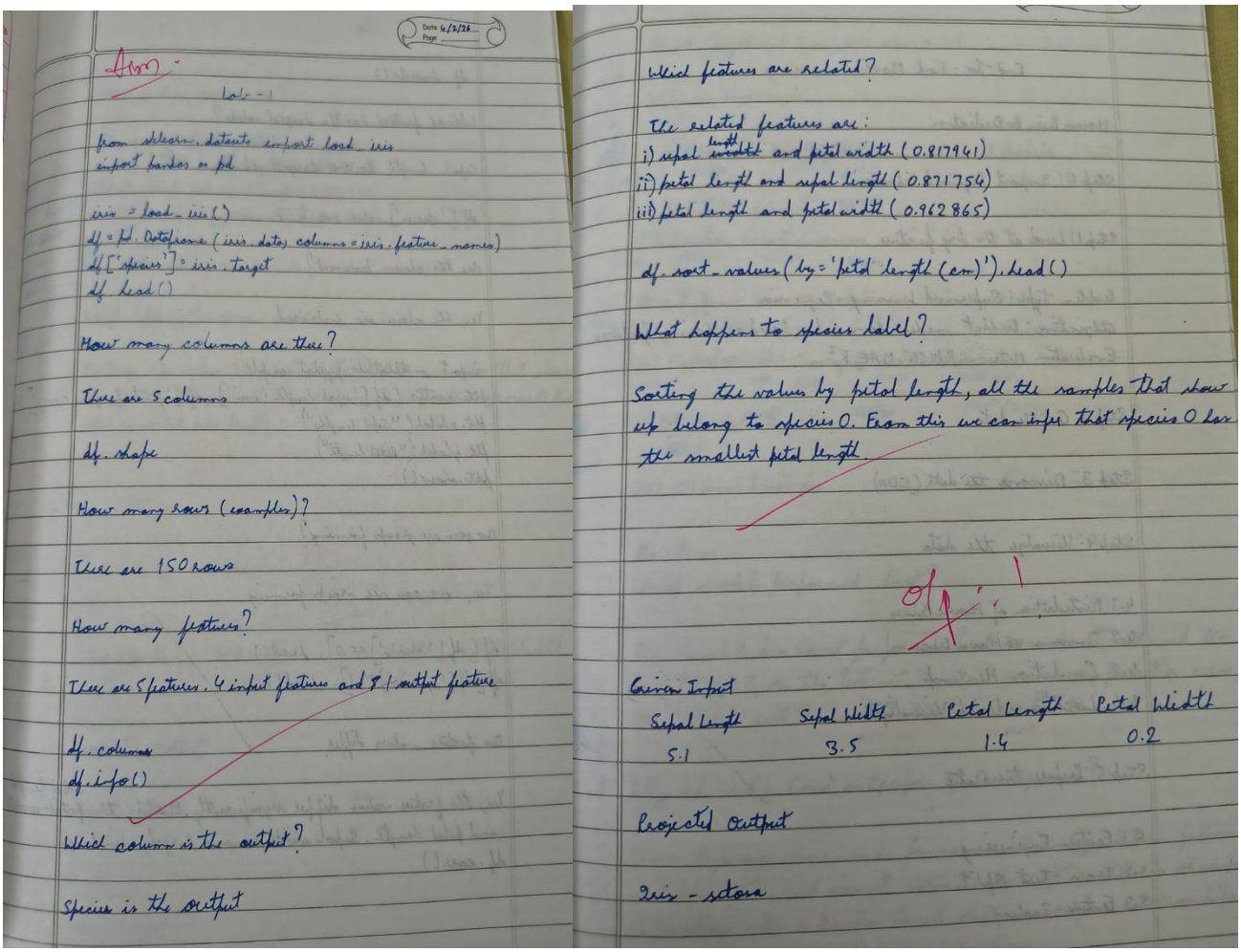
# Index

Github Link:
https://github.com/Crevan-Fernandes/ML

## Program 1

Write a python program to import and export data using Pandas library functions

Screenshot




Code:

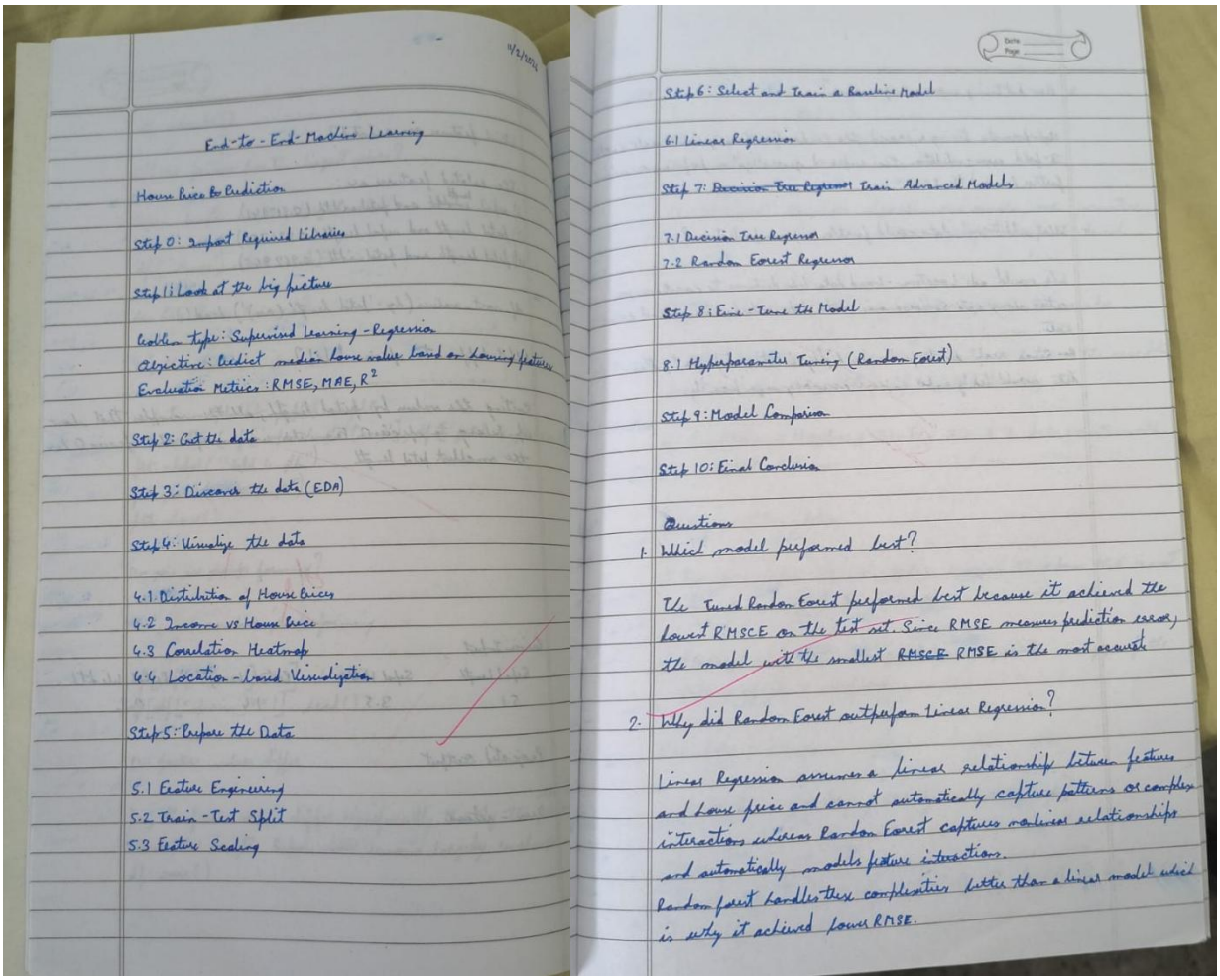
```
import pandas as pd
import numpy as np
def process_storm_data():
try:
df = pd.read_csv('cyclone_mocha_data.csv', na_values=[' ', '-', 'N/A'])
#df = pd.read_excel('coastal_erosion_stats.xlsx', sheet_name='Odisha_2024')
print("--- Data Successfully Imported ---")
print(df.head())
except FileNotFoundError:
```

```
data = {
'Date': ['12.05.23', '12.05.23', '13.05.23'],
'Time_UTC': ['1800', '2100', '0000'],
'Lat': [15.1, 15.2, 15.4],
'Long': [88.8, 88.9, 89.1],
'Central_Pressure_hPa': [954, 954, 954],
'Wind_Speed_kt': [100, 100, 100],
'Grade': ['ESCS', 'ESCS', 'ESCS']
}
df = pd.DataFrame(data)
print("--- Created Sample DataFrame from Scratch ---")
df['Wind_Speed_kmh'] = df['Wind_Speed_kt'] * 1.852
9
df.to_csv('processed_storm_data.csv', index=False)
df.to_json('storm_api_output.json', orient='records', indent=4)
# Export to Excel for stakeholder reports (e.g., NDRF or IMD)
# Requires 'openpyxl' library: pip install openpyxl
df.to_excel('storm_summary_report.xlsx', index=False, sheet_name='Summary')
print("\n--- Data Successfully Exported to CSV, JSON, and Excel ---")
if __name__ == "__main__":
process_storm_data()
```

# Program 2

Demonstrate various data pre-processing techniques for a given dataset

Screenshot




Code:

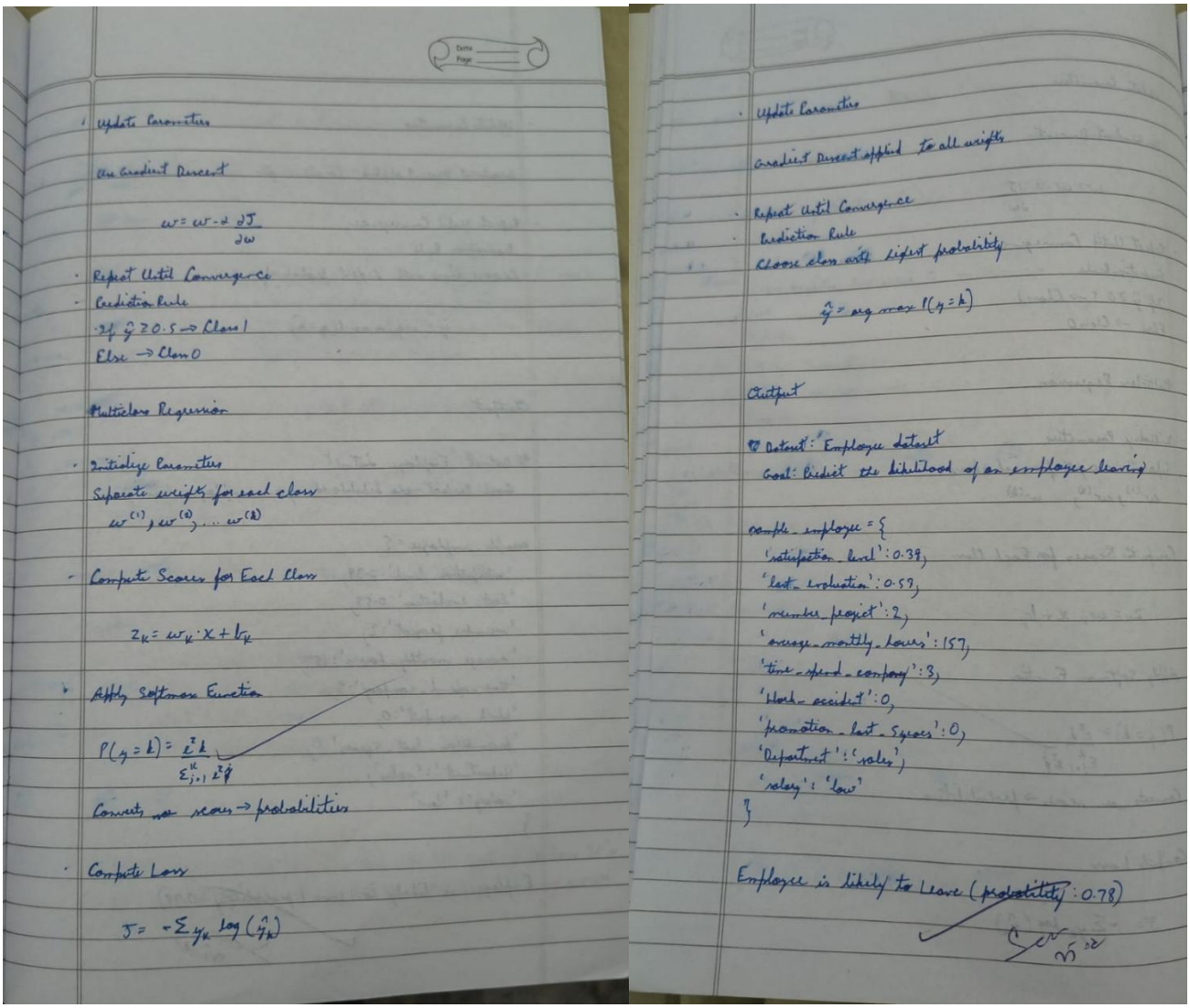
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder
data = {
'Age': [25, 30, 35, np.nan, 200, 22, 38, 45, 12, np.nan], # Contains NaN and Outlier (200)
'Salary': [50000, 54000, np.nan, 62000, 58000, 52000, 120000, 60000, 48000, 55000],
'Department': ['IT', 'HR', 'IT', 'Sales', 'HR', 'IT', np.nan, 'Sales', 'IT', 'HR'],
'Purchased': ['No', 'Yes', 'No', 'Yes', 'No', 'Yes', 'No', 'No', 'No', 'Yes']
}
df = pd.DataFrame(data)
print("Original Dataset:\n", df)
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
```

```
df['Department'] = df['Department'].fillna(df['Department'].mode()[0])
le = LabelEncoder()
df['Purchased'] = le.fit_transform(df['Purchased'])
df = pd.get_dummies(df, columns=['Department'], drop_first=True)
1
5
Q1 = df['Age'].quantile(0.25)
Q3 = df['Age'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
df['Age'] = np.where(df['Age'] > upper_bound, upper_bound,
np.where(df['Age'] < lower_bound, lower_bound, df['Age']))
```

# Program 3

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

Screenshot




Code:

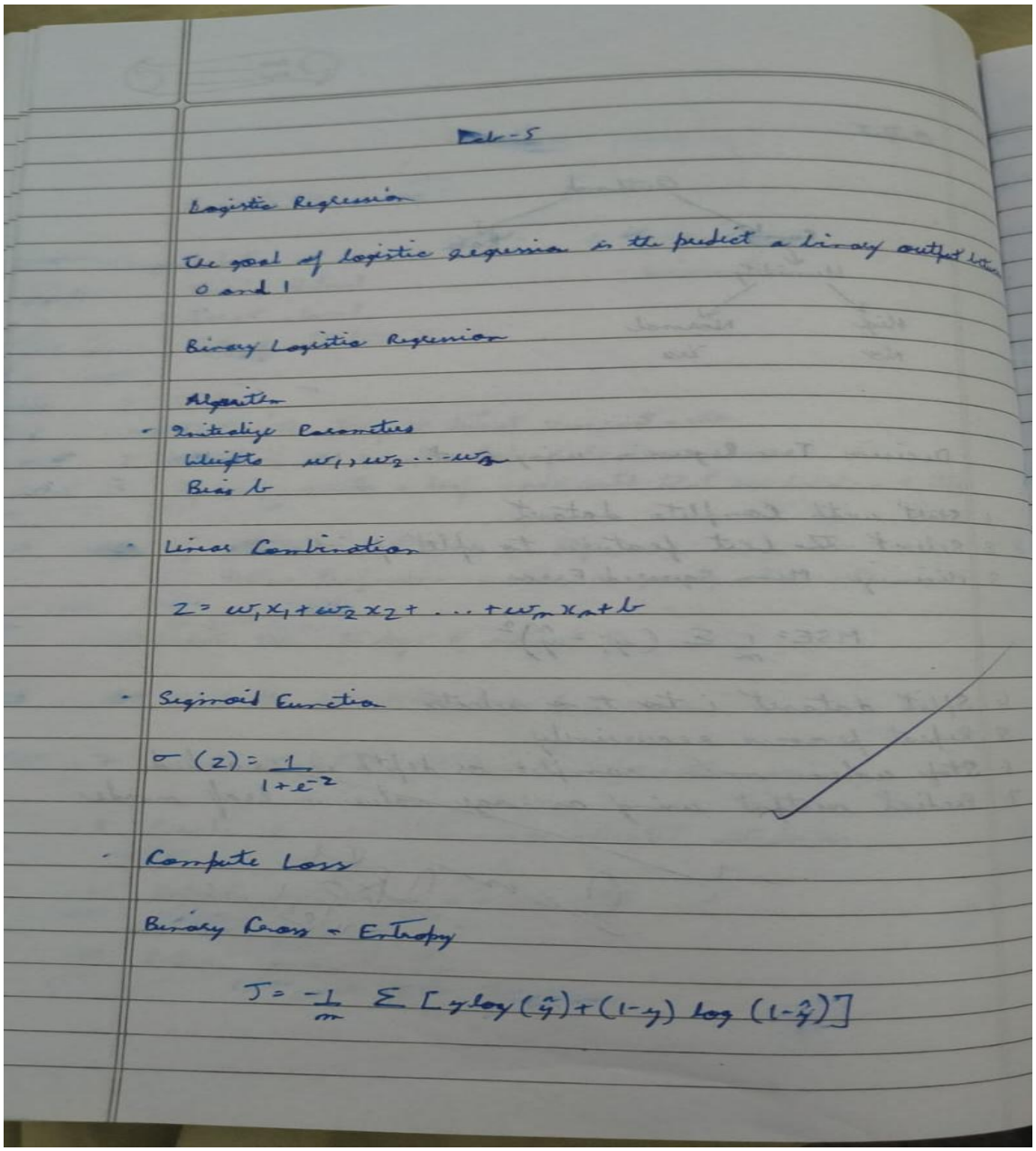
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
df_canada = pd.read_csv('canada_per_capita_income.csv')
X = df_canada[['year']]
y = df_canada['per capita income (US$)']
model = LinearRegression()
model.fit(X, y)
prediction_2020 = model.predict([[2020]])
print(f"Predicted Per Capita Income for 2020: ${prediction_2020[0]:.2f}")
```

```
import pandas as pd
from sklearn.linear_model import LinearRegression
df_salary = pd.read_csv('salary.csv')
df_salary['YearsExperience'] =
df_salary['YearsExperience'].fillna(df_salary['YearsExperience'].median())
X_salary = df_salary[['YearsExperience']]
y_salary = df_salary['Salary']
model_salary = LinearRegression()
model_salary.fit(X_salary, y_salary)
predicted_salary = model_salary.predict([[12]])
print(f"Predicted Salary for 12 years of experience: ${predicted_salary[0]:.2f}")
```

## Program 4

Build Logistic Regression Model for a given dataset

Screenshot




Code:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

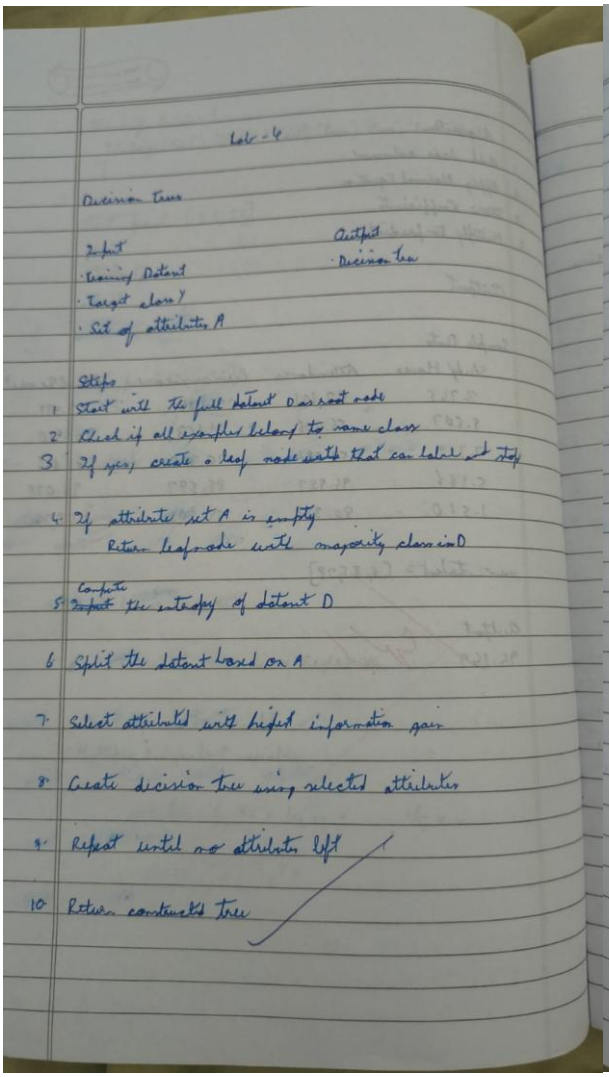
```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# 1. Load the dataset and Preprocessing
# Note: Ensure the csv files are in your working directory
try:
zoo_df = pd.read_csv('zoo-data.csv')
# Preprocessing:
# Usually, 'animal_name' is a unique identifier and should be dropped
# as it provides no predictive value.
if 'animal_name' in zoo_df.columns:
zoo_df = zoo_df.drop('animal_name', axis=1)
# Separate features (X) and target (y)
X = zoo_df.drop('class_type', axis=1)
y = zoo_df['class_type']
# Split into Training (80%) and Testing (20%) sets
2
9
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# 2. Build Logistic Regression Model
# We use multi_class='multinomial' for targets with more than two categories
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
# 3. Measure the accuracy
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy * 100:.2f}%")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
# 4. Plot the Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(10, 7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted Class')
plt.ylabel('Actual Class')
plt.title('Confusion Matrix - Zoo Animal Classification')
plt.show()
3
0
except FileNotFoundError:
print("Error: Please ensure 'zoo-data.csv' is in the same folder as this script.")
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# 1. Load the dataset and Preprocessing
# Note: Ensure the csv files are in your working directory
try:
```

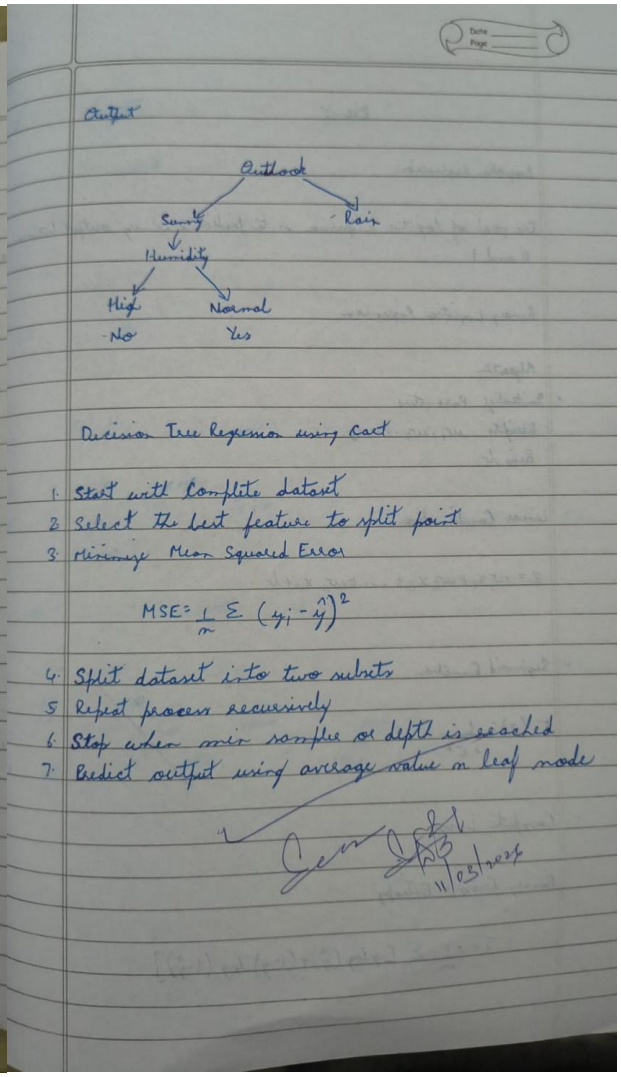
```
zoo_df = pd.read_csv('zoo-data.csv')
# Preprocessing:
# Usually, 'animal_name' is a unique identifier and should be dropped
# as it provides no predictive value.
if 'animal_name' in zoo_df.columns:
zoo_df = zoo_df.drop('animal_name', axis=1)
# Separate features (X) and target (y)
3
1
X = zoo_df.drop('class_type', axis=1)
y = zoo_df['class_type']
# Split into Training (80%) and Testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# 2. Build Logistic Regression Model
# We use multi_class='multinomial' for targets with more than two categories
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
# 3. Measure the accuracy
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy * 100:.2f}%")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
# 4. Plot the Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(10, 7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted Class')
3
2
plt.ylabel('Actual Class')
plt.title('Confusion Matrix - Zoo Animal Classification')
plt.show()
except FileNotFoundError:
print("Error: Please ensure 'zoo-data.csv' is in the same folder as this script.")
```

# Program 5

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshot




Code:

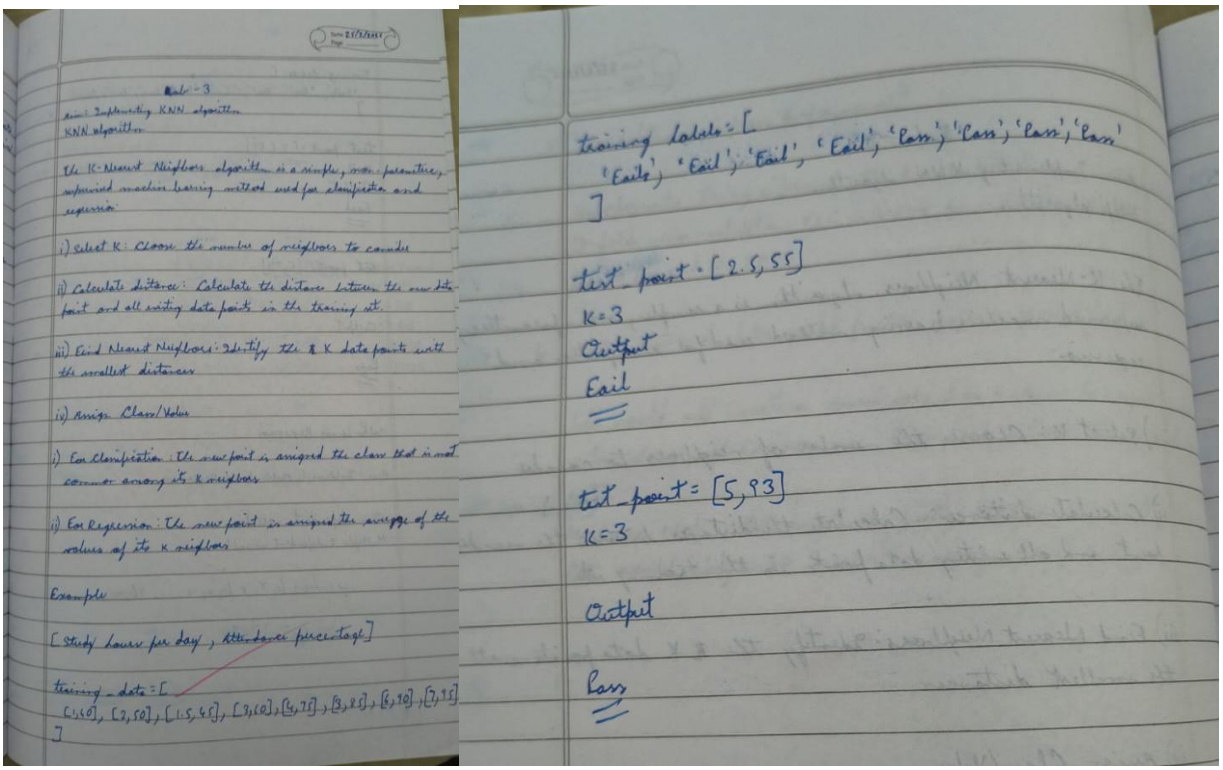
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.preprocessing import LabelEncoder
def build_classification_model(file_path, target_column):
df = pd.read_csv(file_path)
le = LabelEncoder()
for col in df.columns:
X = df.drop(target_column, axis=1)
```

```
y = df[target_column]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
classifier = DecisionTreeClassifier(criterion='entropy')
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
print(f"--- Results for {file_path} ---")
print(f"Accuracy Score: {accuracy_score(y_test, y_pred):.4f}")
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\n")
try:
build_classification_model('iris.csv', 'species')
build_classification_model('drug.csv', 'Drug')
except FileNotFoundError as e:
print(f"Error: {e}")
import pandas as pd
import numpy as np
3
8
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
try:
petrol_df = pd.read_csv('petrol_consumption.csv')
X = petrol_df.drop('Petrol_Consumption', axis=1)
y = petrol_df['Petrol_Consumption']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
regressor = DecisionTreeRegressor()
regressor.fit(X_train, y_train)
y_pred = regressor.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
print("--- Results for Petrol Consumption Regression ---")
print(f"Mean Absolute Error (MAE): {mae:.4f}")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.4f}")
except FileNotFoundError:
print("Error: petrol_consumption.csv not found.")
```

# Program 6

Build KNN Classification model for a given dataset.

Screenshot




Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
iris_df = pd.read_csv('iris.csv')
X = iris_df.drop('species', axis=1)
y = iris_df['species']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print("--- IRIS Dataset Results ---")
print(f"Accuracy Score: {accuracy_score(y_test, y_pred)}")
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

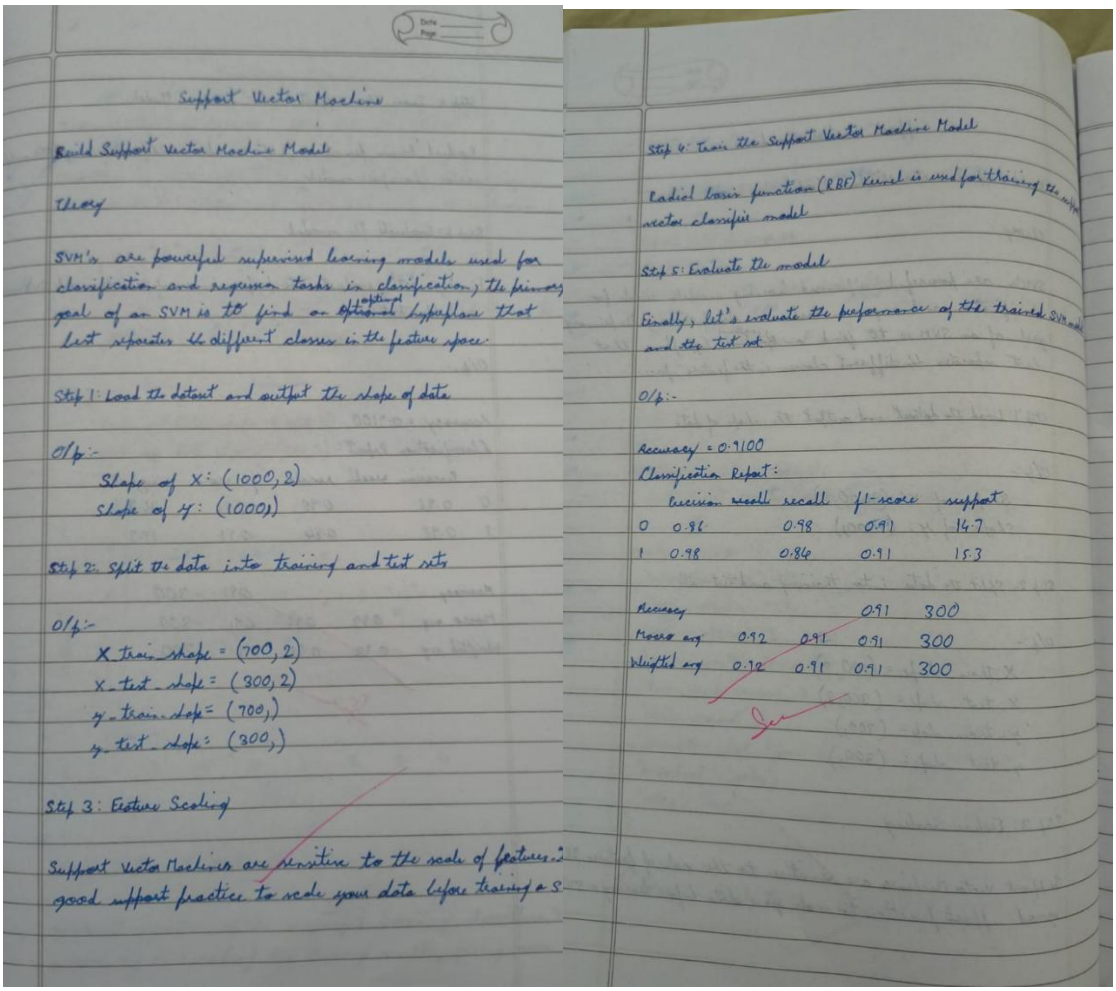
```
diabetes_df = pd.read_csv('diabetes.csv')
cols_fix = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
for col in cols_fix:
diabetes_df[col] = diabetes_df[col].replace(0, diabetes_df[col].median())
X_d = diabetes_df.drop('Outcome', axis=1)
4
2
y_d = diabetes_df['Outcome']
X_train_d, X_test_d, y_train_d, y_test_d = train_test_split(X_d, y_d, test_size=0.2,
random_state=42)
scaler_d = StandardScaler()
X_train_d = scaler_d.fit_transform(X_train_d)
X_test_d = scaler_d.transform(X_test_d)
knn_d = KNeighborsClassifier(n_neighbors=11) #
knn_d.fit(X_train_d, y_train_d)
y_pred_d = knn_d.predict(X_test_d)
print("\n--- Diabetes Dataset Results ---")
print(f"Accuracy Score: {accuracy_score(y_test_d, y_pred_d)}")
print("Confusion Matrix:\n", confusion_matrix(y_test_d, y_pred_d))
import matplotlib.pyplot as plt
import seaborn as sns
heart_df = pd.read_csv('heart.csv')
X_h = heart_df.drop('target', axis=1)
y_h = heart_df['target']
X_train_h, X_test_h, y_train_h, y_test_h = train_test_split(X_h, y_h, test_size=0.2,
random_state=42)
scaler_h = StandardScaler()
X_train_h = scaler_h.fit_transform(X_train_h)
X_test_h = scaler_h.transform(X_test_h)
scores = []
for k in range(1, 21):
knn_h = KNeighborsClassifier(n_neighbors=k)
knn_h.fit(X_train_h, y_train_h)
scores.append(knn_h.score(X_test_h, y_test_h))
4
3
best_k = scores.index(max(scores)) + 1
print(f"\nBest K value found: {best_k} with score: {max(scores)}")
final_knn = KNeighborsClassifier(n_neighbors=best_k)
final_knn.fit(X_train_h, y_train_h)
y_pred_h = final_knn.predict(X_test_h)
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test_h, y_pred_h), annot=True, fmt='d', cmap='Blues')
plt.title(f'Confusion Matrix (K={best_k})')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
report = classification_report(y_test_h, y_pred_h, output_dict=True)
```

```
sns.heatmap(pd.DataFrame(report).iloc[:-1, :].T, annot=True, cmap='YlGnBu')
plt.title('Classification Report')
plt.show()
```

# Program 7

Build Support vector machine model for a given dataset.

Screenshot




Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix
df = pd.read_csv('iris.csv')
X = df.drop('species', axis=1)
y = df['species']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
svm_linear = SVC(kernel='linear')
svm_linear.fit(X_train, y_train)
y_pred_linear = svm_linear.predict(X_test)
print("--- Linear Kernel Results ---")
print(f"Accuracy Score: {accuracy_score(y_test, y_pred_linear)}")
```

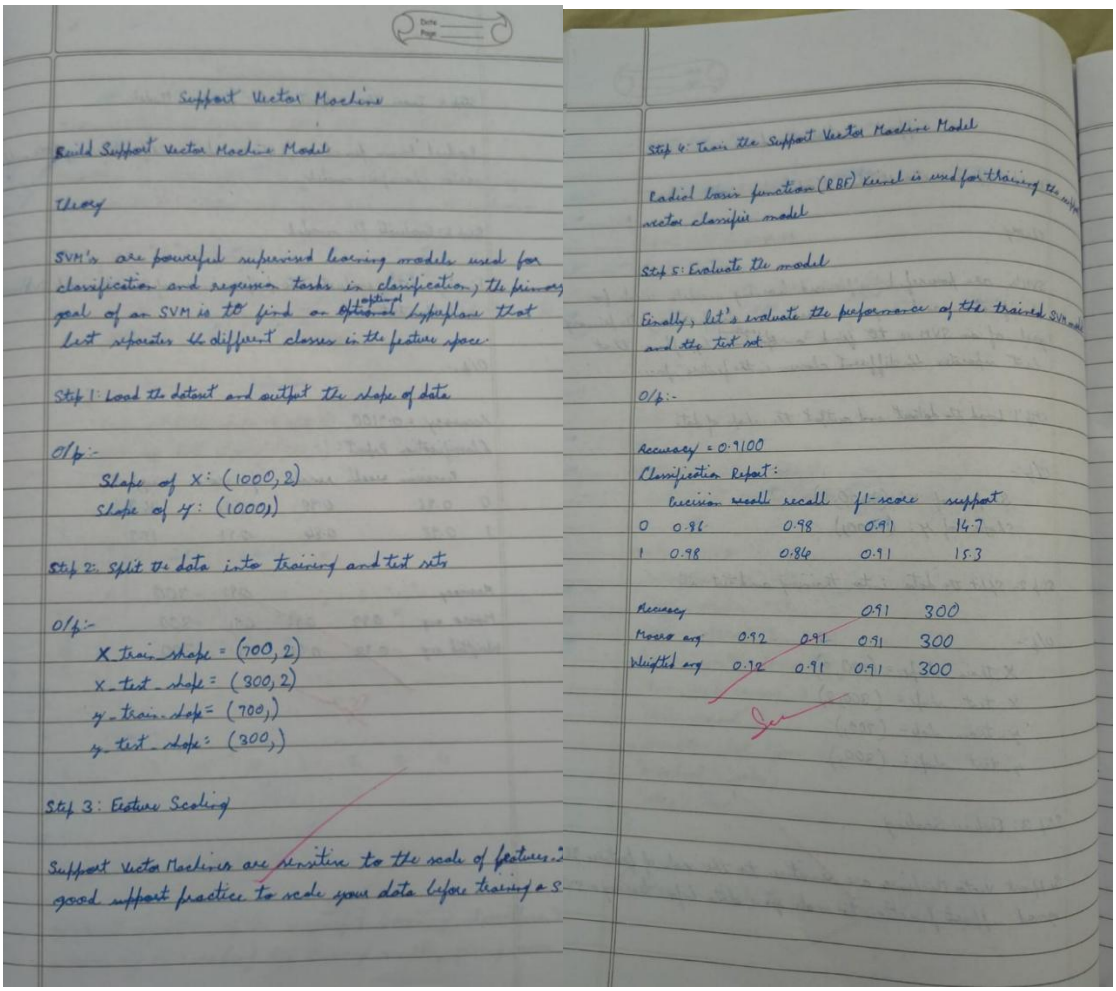
```
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_linear))
svm_rbf = SVC(kernel='rbf')
svm_rbf.fit(X_train, y_train)
y_pred_rbf = svm_rbf.predict(X_test)
4
9
print("\n--- RBF Kernel Results ---")
print(f"Accuracy Score: {accuracy_score(y_test, y_pred_rbf)}")
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_rbf))
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, roc_curve, auc
from sklearn.preprocessing import label_binarize, LabelEncoder
df_letter = pd.read_csv('letter-recognition.csv')
X_l = df_letter.drop('letter', axis=1)
y_l = df_letter['letter']
le = LabelEncoder()
y_enc = le.fit_transform(y_l)
n_classes = len(le.classes_)
X_train_l, X_test_l, y_train_l, y_test_l = train_test_split(X_l, y_enc, test_size=0.2,
random_state=42)
5
0
# 3. Build SVM Classifier (using probability=True for ROC/AUC)
svm_model = SVC(probability=True, random_state=42)
svm_model.fit(X_train_l, y_train_l)
# 4. Predictions and Scores
y_pred_l = svm_model.predict(X_test_l)
y_score_l = svm_model.predict_proba(X_test_l)
print(f"Accuracy Score: {accuracy_score(y_test_l, y_pred_l)}")
print("Confusion Matrix (Partial view):")
print(confusion_matrix(y_test_l, y_pred_l))
# 5. ROC Curve and AUC Score (Micro-average for Multiclass)
y_test_bin = label_binarize(y_test_l, classes=np.arange(n_classes))
fpr, tpr, _ = roc_curve(y_test_bin.ravel(), y_score_l.ravel())
roc_auc = auc(fpr, tpr)
print(f"Micro-average AUC Score: {roc_auc}")
# 6. Plotting
plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
5
1
```

```
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve: Letter Recognition')
plt.legend(loc="lower right")
plt.show()
```

# Program 8

Implement Random forest ensemble method on a given dataset.

Screenshot




Code:

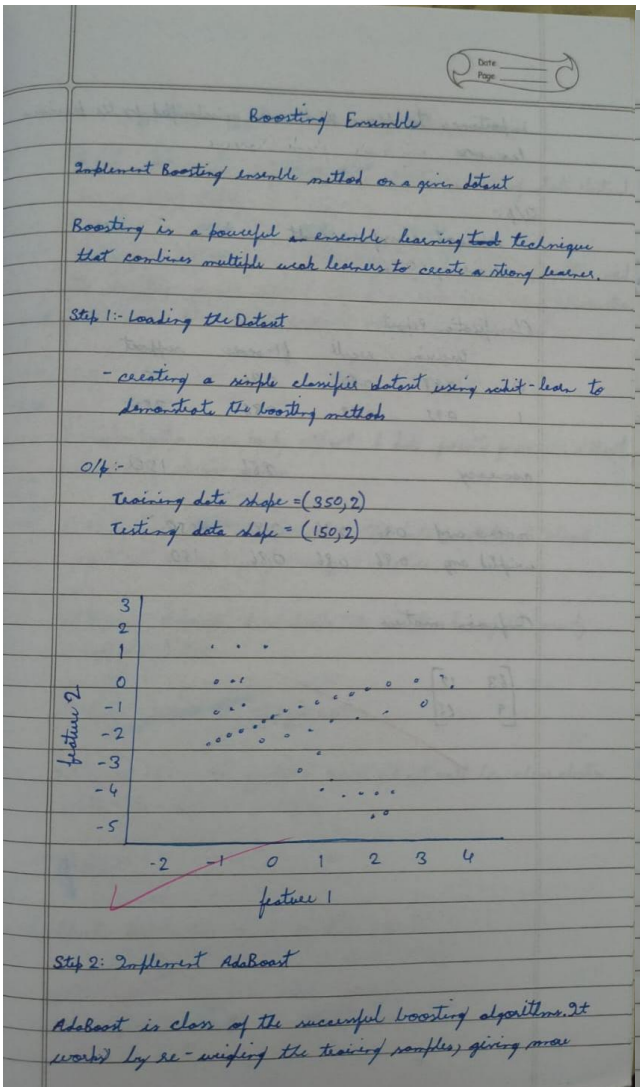
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
df = pd.read_csv('iris.csv')
X = df.drop('species', axis=1)
y = df['species']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
rf_default = RandomForestClassifier(n_estimators=10, random_state=42)
rf_default.fit(X_train, y_train)
y_pred_default = rf_default.predict(X_test)
score_10 = accuracy_score(y_test, y_pred_default)
print(f"Accuracy with n_estimators=10: {score_10 * 100:.2f}%")
```

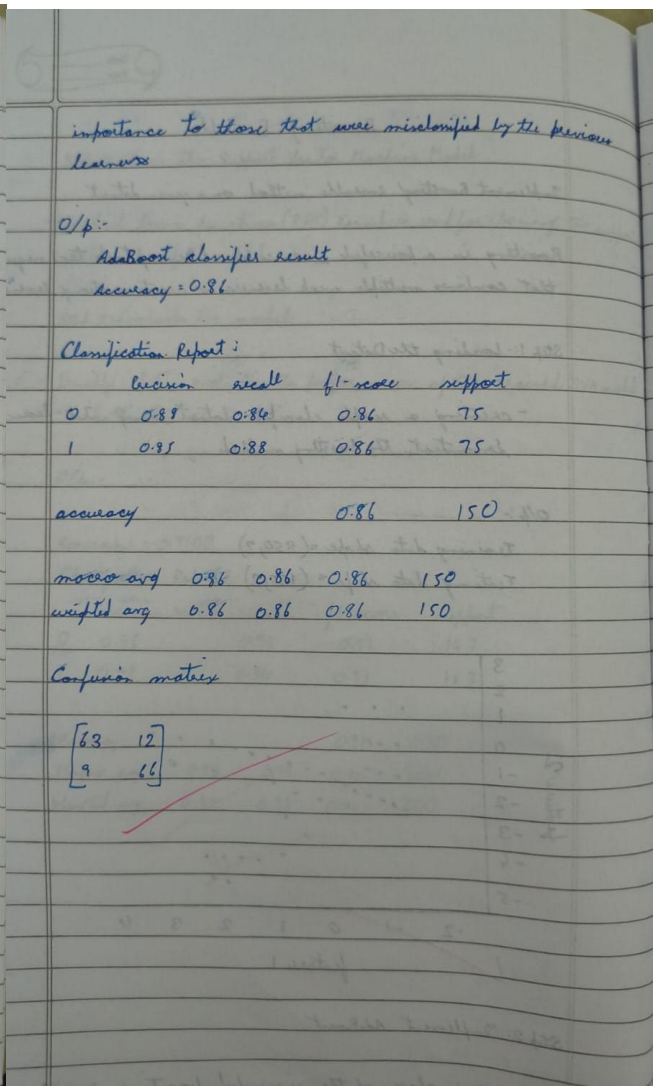
```
best_score = 0
best_n = 0
for n in range(1, 101):
5
6
rf = RandomForestClassifier(n_estimators=n, random_state=42)
rf.fit(X_train, y_train)
score = rf.score(X_test, y_test)
if score > best_score:
best_score = score
best_n = n
print(f"Best Accuracy Score: {best_score * 100:.2f}%")
print(f"Number of trees for best score: {best_n}")
```

# Program 9

Implement Boosting ensemble method on a given dataset.

Screenshot




Code:

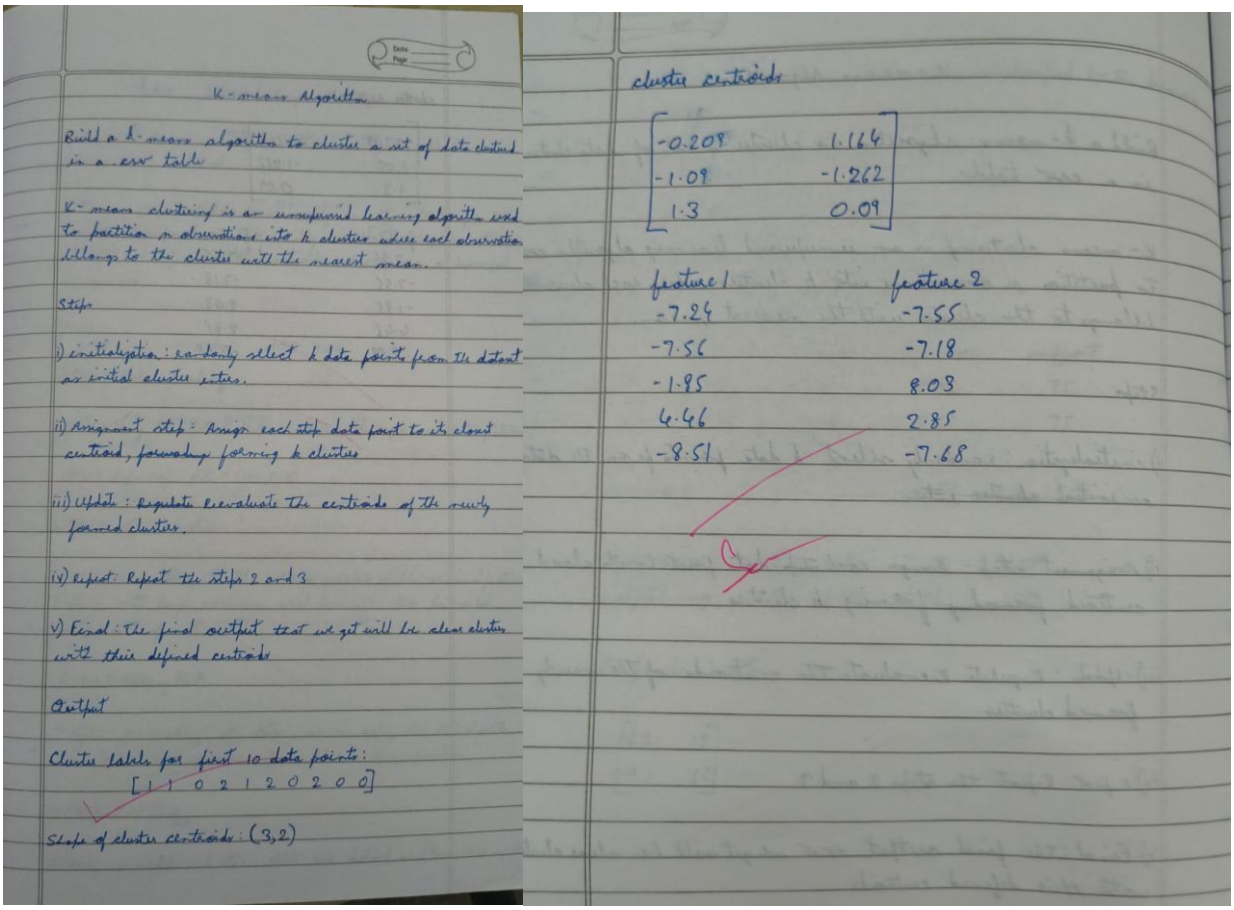
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score
df = pd.read_csv('income.csv')
# Separate features (X) and target (y)
X = df.drop('income_level', axis=1)
y = df['income_level']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
ada_10 = AdaBoostClassifier(n_estimators=10, random_state=42)
ada_10.fit(X_train, y_train)
y_pred_10 = ada_10.predict(X_test)
score_10 = accuracy_score(y_test, y_pred_10)
print(f"Prediction score with 10 trees: {score_10:.4f}")
# Testing various numbers of trees to identify the best performance
tree_counts = [5, 10, 25, 50, 100, 150, 200, 300, 400, 500]
scores = []
print("\nFine-tuning results:")
for n in tree_counts:
model = AdaBoostClassifier(n_estimators=n, random_state=42)
model.fit(X_train, y_train)
acc = model.score(X_test, y_test)
scores.append(acc)
print(f"Trees: {n:3} | Accuracy: {acc:.4f}")
best_acc = max(scores)
best_n = tree_counts[scores.index(best_acc)]
6
1
print(f"\nOptimal performance: {best_acc:.4f} accuracy achieved with {best_n} trees.")
# 5. Visualization (Optional but helpful)
plt.figure(figsize=(8, 5))
plt.plot(tree_counts, scores, marker='o', linestyle='-', color='darkgreen')
plt.title('AdaBoost Accuracy vs. Number of Trees')
plt.xlabel('n_estimators')
plt.ylabel('Test Accuracy')
plt.grid(True)
plt.show()
```

# Program 10

Build k-Means algorithm to cluster a set of data stored in a .CSV file.

Screenshot




Code:

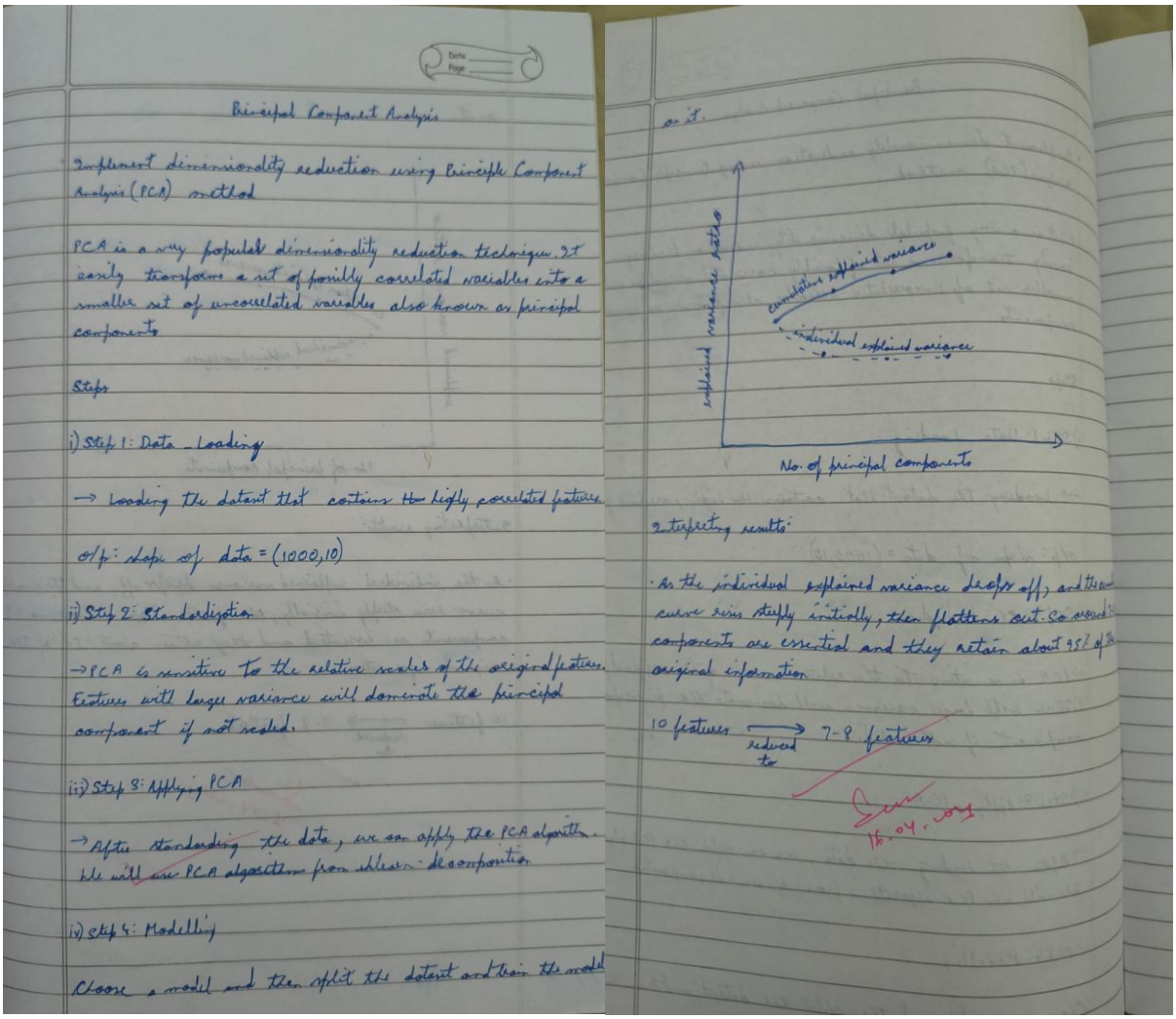
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.decomposition import PCA
df = pd.read_csv('heart.csv')
categorical_features = ['cp', 'restecg', 'slope', 'ca', 'thal']
df_encoded = pd.get_dummies(df, columns=categorical_features, drop_first=True)
X = df_encoded.drop('target', axis=1)
y = df_encoded['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
6
7
models = {
"Logistic Regression": LogisticRegression(random_state=42),
"SVM": SVC(random_state=42),
"Random Forest": RandomForestClassifier(random_state=42)
}
print("--- Accuracies Before PCA ---")
results_before = {}
for name, model in models.items():
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
acc = accuracy_score(y_test, y_pred)
results_before[name] = acc
print(f"{name}: {acc:.4f}")
pca = PCA(n_components=0.95)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
print(f"\nOriginal dimensions: {X_train_scaled.shape[1]}")
print(f"Dimensions after PCA: {X_train_pca.shape[1]}")
print("\n--- Accuracies After PCA ---")
6
8
results_after = {}
for name, model in models.items():
model.fit(X_train_pca, y_train)
y_pred = model.predict(X_test_pca)
acc = accuracy_score(y_test, y_pred)
results_after[name] = acc
print(f"{name}: {acc:.4f}")
```

# Program 11

Implement Dimensionality reduction using Principal Component Analysis (PCA) method.

Screenshot




Code:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.decomposition import PCA
df = pd.read_csv('heart.csv')
categorical_cols = ['cp', 'restecg', 'slope', 'ca', 'thal']
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)
```

```
X = df_encoded.drop('target', axis=1)
y = df_encoded['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
7
4
def evaluate_models(X_tr, X_te, y_tr, y_te):
models = {
"SVM": SVC(),
"Logistic Regression": LogisticRegression(),
"Random Forest": RandomForestClassifier(random_state=42)
}
accuracies = {}
for name, model in models.items():
model.fit(X_tr, y_tr)
y_pred = model.predict(X_te)
accuracies[name] = accuracy_score(y_te, y_pred)
return accuracies
results_initial = evaluate_models(X_train_scaled, X_test_scaled, y_train, y_test)
print("--- Accuracies Without PCA ---")
for model, acc in results_initial.items():
print(f"{model}: {acc:.4f}")
pca = PCA(n_components=0.95)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
print(f"\nOriginal dimensions: {X_train_scaled.shape[1]}")
7
5
print(f"Dimensions after PCA: {X_train_pca.shape[1]}")
results_pca = evaluate_models(X_train_pca, X_test_pca, y_train, y_test)
print("\n--- Accuracies With PCA ---")
for model, acc in results_pca.items():
print(f"{model}: {acc:.4f}")
```